

UNISOLE AI CAMPUS PROGRAM

GROUP 1 — COMPUTER SCIENCE & IT

Pathway 1: Machine Learning Engineering in Production

Target students: BCA • MCA • B.Sc. CS/IT • PGDCA • B.Tech CSE/IT • Related disciplines

Duration: 3 Months

Recommended level: Intermediate / students with basic Python exposure

Format: Weekday technical track (can be paired with the weekend AI Entrepreneurship & Innovation track)

Overview

This pathway is built for one purpose: to take a student from writing basic scripts to shipping a real, working AI product. Most AI courses stop at "train a model in a notebook." This one doesn't — it is sequenced so every module unlocks the next, and it ends with a deployed, monitored system rather than a slideshow. Students who complete this pathway walk away able to explain, defend, and demonstrate a full AI product lifecycle: data → model → API → application → deployment.

Why this pathway matters: Employers increasingly don't just want someone who can build a model — they want someone who can take that model and turn it into software other people can actually use. This pathway is deliberately structured around that gap.

Module 1 — Python for AI Engineering

This module builds the coding foundation every later module depends on. Rather than teaching Python as an isolated language, it is taught as an engineering tool — how professionals structure, reuse, and share code so it survives beyond a single script.

Topics covered:

- Python programming fundamentals — syntax, data types, control flow
- Functions, modules and packages — writing code that can be reused, not just run once
- Object-oriented programming — structuring larger applications around classes and objects
- Exception handling — building code that fails gracefully instead of crashing
- File and data handling — reading, writing and processing real files
- Virtual environments — isolating project dependencies the way professional teams do
- Package management — installing, managing, and pinning library versions
- Writing reusable code — functions and modules designed to be imported elsewhere
- Git/GitHub workflow — the version control system used by virtually every software team

Skills gained: Confident, professional-grade Python coding; comfort with the command line and Git; the ability to read and extend someone else's codebase.

Practical: Build a small Python application and publish it on GitHub.

Module 2 — Data Engineering Foundations

Before any model can be trained, data has to be sourced, cleaned, and organized. This module teaches students how real companies move and store data — from raw files to queryable warehouses — so later ML work starts from clean, trustworthy inputs rather than guesswork.

Topics covered:

- Structured vs unstructured data — recognizing what format data arrives in and why it matters
- CSV, JSON and Parquet — the three file formats data actually moves in
- NumPy — fast numerical computation on large arrays
- Pandas — the standard toolkit for cleaning and exploring tabular data
- Data cleaning and validation — handling missing values, duplicates, and inconsistent data
- Exploratory data analysis — understanding a dataset before modelling it
- SQL fundamentals and relational databases — querying data directly from a database
- OLTP vs OLAP — transactional systems vs analytical systems
- ETL vs ELT — the two dominant patterns for moving and transforming data
- Batch vs streaming data — data that arrives all at once vs continuously
- DuckDB — a fast, modern analytical database for local data processing
- Apache Kafka fundamentals — how companies stream real-time data at scale

Skills gained: The ability to take messy, real-world data and turn it into something a model can actually be trained on; comfort with SQL and modern data tools.

Practical: Data Source → ETL → Parquet → DuckDB → Analytics / ML

Module 3 — Machine Learning

This is the core statistical-modelling module. Students learn how to frame a business problem as a learning problem, choose the right algorithm, and — just as importantly — measure whether the resulting model is actually good enough to trust.

Topics covered:

- ML workflow — the standard end-to-end process every ML project follows
- Supervised learning — learning from labelled examples
- Regression and Classification — predicting numbers vs predicting categories
- Unsupervised learning and Clustering — finding structure in unlabelled data
- Feature engineering — transforming raw data into inputs a model can learn from
- Train/validation/test split and Cross-validation — evaluating a model honestly
- Model evaluation — Accuracy, Precision, Recall, F1-score, Confusion matrix
- Model selection — choosing between competing algorithms with evidence, not guesswork

Skills gained: The ability to build, evaluate, and defend a machine learning model, and to explain in plain language why one model was chosen over another.

Module 4 — Deep Learning & Modern AI

Once classical ML is solid, students move into the architectures that power today's AI systems — from image-recognition networks to the transformer models behind large language models.

Topics covered:

- Neural networks and Forward/backpropagation concepts — how a network actually learns
- CNN fundamentals — the architecture behind computer vision
- Sequence models overview — handling data that comes in order, like text or time series
- Transfer learning and Model fine-tuning concepts — adapting a pretrained model instead of starting from scratch
- Introduction to Transformers — the architecture behind modern LLMs
- Embeddings — representing meaning as numbers
- LLM fundamentals — how large language models are built and how they behave

Skills gained: A working understanding of the AI architectures behind almost every modern AI product, and the vocabulary to keep learning independently as the field moves.

Module 5 — Building AI Applications

A model sitting in a notebook is not useful until it can be called by real software. This module teaches students to wrap a trained model in a production-style API that any application can talk to.

Topics covered:

- Model inference — running a trained model to get predictions
- REST APIs — the standard way software systems talk to each other
- FastAPI — a modern, high-performance Python framework for building APIs
- Request/response validation and Pydantic — making sure an API only accepts correct data
- Connecting models to applications — the actual "plumbing" that makes AI usable
- Authentication fundamentals — controlling who can access an API
- Error handling — building an API that fails safely and informatively

Skills gained: The ability to turn a trained model into a real, callable service — the single skill that separates "I trained a model" from "I built a product."

Module 6 — MLOps & Deployment

Models degrade and systems break — this module covers the operational discipline that keeps an AI system reliable after launch, not just on the day it's built.

Topics covered:

- Model serialization — saving a trained model so it can be reused
- Experiment tracking and Model/version management — keeping track of what was tried and what worked
- Docker and Containerization — packaging an application so it runs identically anywhere
- CI/CD fundamentals — automating testing and deployment
- Cloud deployment concepts — getting an application live and accessible
- Monitoring — watching a live system for problems
- Data drift and Model drift — recognizing when a model's real-world performance starts degrading
- Retraining pipelines — keeping a model up to date over time

Skills gained: The operational maturity to keep an AI system running reliably in production — a skill most bootcamp-level courses skip entirely.

Module 7 — Generative AI & RAG

The most in-demand AI skill today. Students learn to build applications on top of large language models, including Retrieval-Augmented Generation (RAG) systems that let an LLM answer questions using a company's own documents.

Topics covered:

- LLM architecture overview — how large language models are structured
- Prompt engineering — getting reliable, high-quality output from an LLM
- Embeddings and Vector databases — representing and searching meaning, not just keywords
- RAG architecture — combining retrieval with generation for grounded, accurate answers
- Document ingestion and Chunking — preparing documents for retrieval
- Retrieval and LLM response generation — the two halves of a RAG pipeline
- AI application evaluation — measuring whether an AI feature is actually good
- Agent fundamentals — AI systems that can take multi-step actions, not just answer questions

Skills gained: The ability to build the kind of AI-powered features (chatbots, document Q&A, AI assistants) that are currently the fastest-growing category of software product.

Capstone Project

Students build a complete, end-to-end AI system that integrates every module into one working product:

Data → Processing → ML/AI Model → API → Application → GitHub → Docker → Deployment

This is not a toy exercise — it mirrors the actual lifecycle a junior ML engineer would be expected to own at a real company, from raw data to a live, deployed application.

Tools & technologies used across this pathway: Python, Git/GitHub, Pandas, NumPy, SQL, DuckDB, scikit-learn-style ML workflow, FastAPI, Pydantic, Docker, vector databases, LLM APIs.

Career relevance: ML Engineer, AI Engineer, Applied ML Developer, Data Scientist (engineering-leaning), Backend Developer with AI specialization.

Student outputs: GitHub repository, ML project, API, deployed application where applicable, technical documentation, project presentation, resume project entry

Industry-Readiness Layer (*applies alongside this pathway*)

Professional Digital Profile: Professional email, LinkedIn, Resume, Portfolio, Project documentation, Professional communication.

GitHub / Evidence: Git, GitHub, README, Repository structure, Version control — the way employers expect a technical portfolio to be presented.

Interview Preparation: Resume walkthrough, Tell-me-about-yourself, Project explanation, Behavioural questions, Technical/domain questions, Mock interview, Feedback.

Industry Interaction: Weekly/periodic expert sessions, Career sessions, Industry case studies, How companies recruit, How to approach professionals, Networking etiquette.

Project Defence: Every student must be able to answer — What problem did you solve? Why did you choose this approach? What did you build? What did you learn? What are the limitations? What would you improve?

Final Capstone Framework

1. **Problem** — Identify a real-world problem
2. **Research** — Understand the domain and existing solutions
3. **Data/Information** — Collect and organize relevant information
4. **Build** — Create the appropriate technical solution
5. **Validate** — Test the solution and interpret results
6. **Document** — Report, Repository/portfolio, Documentation, Presentation
7. **Defend** — Present the project before an evaluator/mentor

What This Student Should Have at the End

GitHub + Software Project + AI/ML Project + Deployment where applicable + Resume

Training and assessment are program components. Internship, advanced projects, hackathons, research engagement and other opportunities are performance/availability-based and subject to applicable selection procedures.